

Encoding & Embedding

Session Notes — AI Engineering Backlog

Topics: Encoding · Mathematical Intuition · Practical Implementation · Word2Vec Intro

❖ 1. What is Encoding vs Embedding?

Encoding → converts text to numbers. No meaning preserved.

Example: cat → [1, 0, 0]

Embedding → converts text to numbers. Meaning preserved.

Example: cat → [0.21, -0.45, 0.87, ...]

Encoding asks: *what words exist?*

Embedding asks: *what do these words mean?*

Scalar · Vector · Matrix

Scalar → single number → 5

Vector → list of numbers → [5, 10]

Matrix → grid of numbers → [[5, 10], [2, 4]]

❖ 2. Encoding Methods

One-Hot Encoding

Each word → one **1**, rest **0s**. Vocabulary size = vector size.

Vocab: [cat, sat, mat]

cat → [1, 0, 0]

sat → [0, 1, 0]

```
mat → [0, 0, 1]
```

■ 10,000 word vocab = 10,000 dimension vector (sparse) · No meaning

Bag of Words (BOW)

Count how many times each word appears. Loses word order.

"Dog bites man" → [bites:1, dog:1, man:1]

"Man bites dog" → [bites:1, dog:1, man:1] ← **IDENTICAL!**

Completely different meaning — same vector. BOW cannot tell them apart.

Library: CountVectorizer from sklearn

❖ 3. TF-IDF — Term Frequency · Inverse Document Frequency

Fixes BOW: common words like 'the', 'is' get low weight. Rare but important words get high weight.

TF = count of word in this doc / total words in this doc

IDF = $\log(\text{total documents} / \text{documents containing this word})$

TF-IDF = TF × IDF

Why log? Without it, a word appearing 100× seems 10× more important than one appearing 10×. Log dampens extremes.

'the' appears in every doc

→ IDF near 0

→ **low weight**

'RBI' appears in 2 finance docs

→ IDF is high

→ **high weight**

★ **Rule:** Use `fit_transform()` on training data only. Use `transform()` on new/test data — never fit on test data.

`fit_transform()` = `fit()` + `transform()` in one step

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer()

X = tfidf.fit_transform(docs) # learn + convert

print(X.toarray()) # see numbers

print(tfidf.get_feature_names_out()) # see vocab
```

❖ 4. Keyword Search vs Semantic Search

Keyword Search

Exact word match

Uses: TF-IDF / BM25

Query: "affordable loan"

Finds: "affordable loan" only

Misses: "cheap mortgage"

Semantic Search

Meaning match

Uses: Embeddings + Vector DB

Query: "affordable loan"

Finds: "cheap mortgage" too ✓

Understands context

Real RAG uses both — called Hybrid Search

BM25 for exact matches + Embeddings for semantic matches

❖ 5. Word2Vec — Where Meaning Begins

First method to capture **semantic meaning** in vectors. Features are **learned from data**, not hand-crafted.

king → [0.50, 0.21, -0.12, ...]

queen → [0.48, 0.19, -0.11, ...] ← very similar

dog → [0.91, -0.44, 0.87, ...] ← far from king

Famous example: king – man + woman ≈ queen

CBOW

Continuous Bag of Words

Predict center word from context

Skip-gram

Predict context words from center word

Better for rare words

The Progression

One-Hot → huge, sparse, no meaning

↓

BOW/TF-IDF → smaller, still no meaning, loses order

↓

Word2Vec → dense, captures meaning, one vector/word

↓

Transformers → dense, context-aware, different vector per context

↓

SOTA Models → what RAG and Vector DBs use today

❖ 6. Mini-Project — Build a Similarity Search

```
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

docs = [

    "RBI sets repo rate to control inflation",

    "SEBI regulates the stock market in India",

    "SIP allows monthly investment in mutual funds",

    "NSE and BSE are major stock exchanges",
```

```
"ELSS qualifies for Section 80C deduction"
```

```
]
```

```
query = "how to save tax in India"
```

```
tfidf = TfidfVectorizer()
```

```
vecs = tfidf.fit_transform(docs)
```

```
qvec = tfidf.transform([query])
```

```
sims = cosine_similarity(qvec, vecs)
```

```
print(docs[sims.argmax()])
```

Quick Reference

Scalar = single number · **Vector** = list of numbers · **Matrix** = grid

Encoding = numbers without meaning → sparse vectors

Embedding = numbers with meaning → dense vectors

CountVectorizer → Bag of Words

TfidfVectorizer → TF-IDF

Keyword search = exact word match → BM25

Semantic search = meaning match → Vector DB + embeddings